



MOBILE APPLICATION DEVELOPMENT

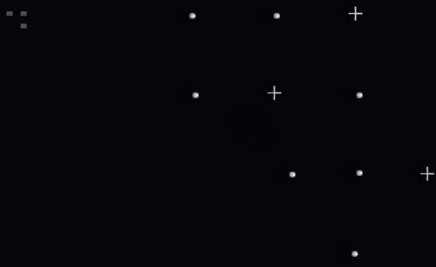
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS



AULA 03

Alterando dados remotos

Como uma ação do usuário muda o dado no servidor



“*Nada é permanente, exceto a mudança.*”
— *Heráclito*

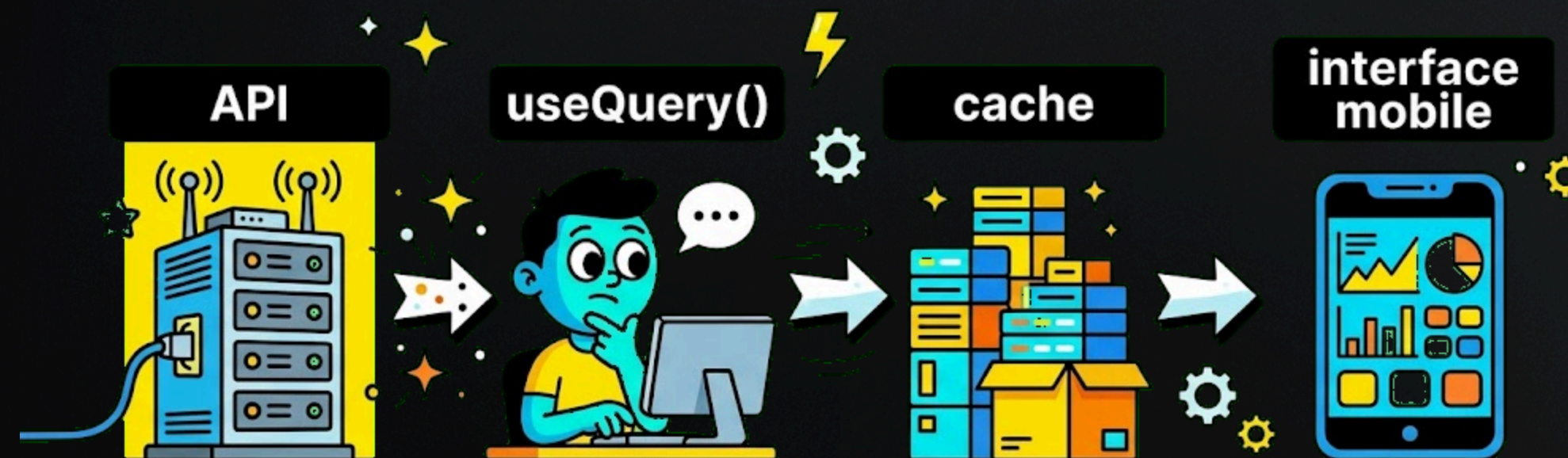




Mentimeter

ATÉ AGORA...

Nossa aplicação consegue:



NÓS APRENDEMOS A LER DADOS REMOTOS.

MAS AGORA TEMOS OUTRO PROBLEMA

Nossa aplicação consegue listar tweets.



COMO ADICIONAMOS UM NOVO TWEET?

UMA AÇÃO DO USUÁRIO

Imagine um botão:

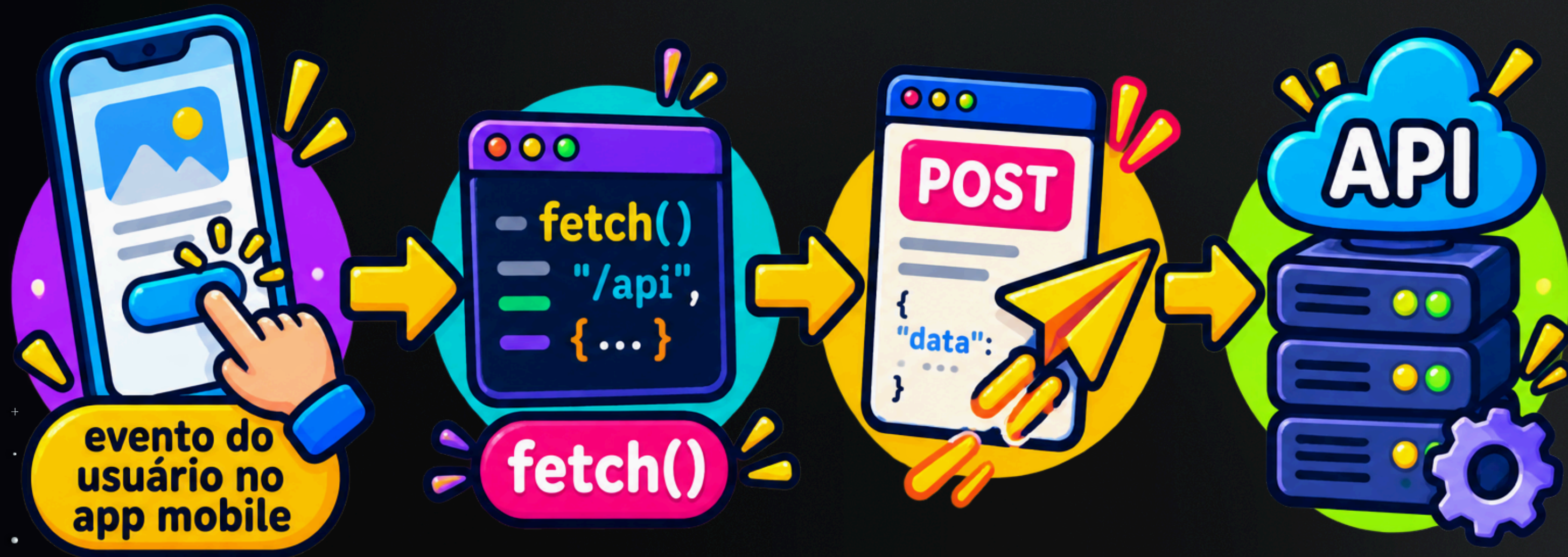


Ao tocar nele...

ALGUMA COISA PRECISA ACONTECER NO SERVIDOR.

ANTES DO TANSTACK QUERY

Nós já sabemos fazer isso.



CRIANDO UM TWEET

```
type NewTweet = {  
  text: string;  
};  
  
async function createTweet(tweet: NewTweet) {  
  const response = await fetch(`${API_URL}/tweets`, {  
    method: "POST",  
    headers: {  
      "Content-Type": "application/json",  
    },  
    body: JSON.stringify(tweet),  
  });  
  
  if (!response.ok) {  
    throw new Error("Não foi possível adicionar o tweet");  
  }  
  
  return response.json();  
}
```


QUANDO EXECUTAR?

```
async function handleAddTweet() {  
  await createTweet({  
    text: "Nostalgia...",  
  });  
}
```

```
<Pressable onPress={handleAddTweet}>  
  <Text>Adicionar tweet</Text>  
</Pressable>
```

O POST FUNCIONA.

MAS...

O que deveria acontecer enquanto o `POST` está sendo executado?

E se der erro?

E quando terminar?

+
| Podemos adicionar outro tweet enquanto o primeiro está sendo salvo?

PODEMOS CONTROLAR MANUALMENTE

```
const [isSaving, setIsSaving] = useState(false);
const [error, setError] = useState<Error | null>(null);

async function handleAddTweet() {
  try {
    setIsSaving(true);
    setError(null);

    await createTweet(tweet);
  } catch (error) {
    setError(error as Error);
  } finally {
    setIsSaving(false);
  }
}
```


ISSO PARECE FAMILIAR?

Quando fizemos uma consulta manualmente, também precisávamos controlar:

- dados;
- carregamento;
- erro;
- sucesso.

DEPOIS CONHECEMOS O USEQUERY.

DOIS TIPOS DE OPERAÇÃO

QUERY

Queremos **obter** dados.

GET /tweets

O servidor é nossa
fonte de informação.

MUTATION

Queremos **causar uma mudança**.

POST
PUT
PATCH
DELETE

Existe uma ação a ser executada.

MUTATION

Uma mutation representa uma operação que provoca uma mudança.

Por exemplo:

```
criar um filme  
alterar um pedido  
excluir um comentário  
confirmar uma reserva  
publicar uma mensagem
```

CADA AÇÃO REPRESENTA UMA MUTATION DIFERENTE.

Criar, alterar e excluir são operações diferentes.

Cada uma possui sua própria mutationFn e seu próprio ciclo

UMA MUTATION PARA CADA AÇÃO

```
// criar filme
useMutation({ mutationFn: criarFilme });

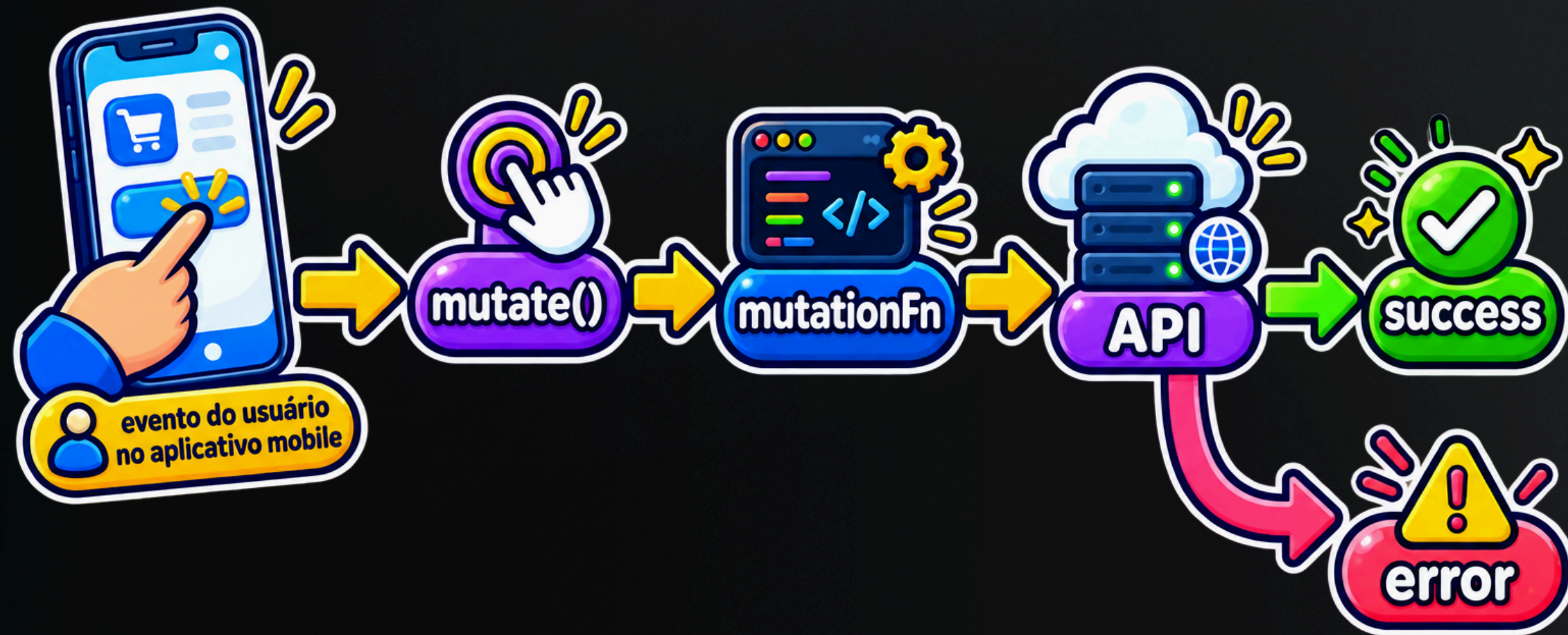
// alterar um pedido
useMutation({ mutationFn: atualizarPedido });

// excluir um comentário
useMutation({ mutationFn: excluirComentario });
```

Criamos mutations para as ações que podem acontecer.

O MODELO MENTAL

Antes do código:



QUEM É QUEM?

`mutate()`

Quando executar a operação.

Normalmente chamado após uma ação do usuário.

`mutationFn`

O que precisa ser executado.

A função assíncrona que conversa com a API.

AGORA COM USEMUTATION

```
const createTweetMutation = useMutation({  
  mutationFn: createTweet,  
});
```

Ainda não aconteceu nada.

CRIAMOS A MUTATION.

QUEM DISPARA A OPERAÇÃO?

```
createTweetMutation.mutate({  
  text: "Matrix",  
});
```

Agora:

```
mutate(tweet)  
  ↓  
createTweet(tweet)  
  ↓  
POST /tweets
```


NO BOTÃO

```
function handleAddTweet() {  
  createTweetMutation.mutate({  
    text: "Café e código, a melhor combinação.",  
  });  
}
```

```
<Pressable onPress={handleAddTweet}>  
  <Text>Adicionar tweet</Text>  
</Pressable>
```


QUERY × MUTATION

```
useQuery({  
  queryKey: ["tweets"],  
  queryFn: fetchTweets,  
});
```

A query pode executar porque queremos **observar dados**.


```
useMutation({  
  mutationFn: createTweet,  
});
```

A mutation espera alguém dizer:

```
mutate(tweet);
```

ELA REPRESENTA UMA AÇÃO.

MUTATION STATES

O QUE ESTÁ ACONTECENDO COM A OPERAÇÃO?

CICLO DE UMA MUTATION



Uma mutation está em apenas um desses estados por vez.

ESTADOS DISPONÍVEIS

isIdle

A operação ainda não começou.

isPending

A operação está sendo executada.

isSuccess

A operação terminou com sucesso.

isError

A operação terminou com erro.

AGORA A UI SABE O QUE ACONTECEU

```
const { mutate, isPending, isError, isSuccess, error } = useMutation({  
  mutationFn: createTweet,  
});
```


ENQUANTO ESTAMOS SALVANDO

```
<Pressable
  disabled={isPending}
  onPress={() =>
    mutate({
      text: "Café e código, a melhor combinação.",
    })
  }
>
  <Text>{isPending ? "Salvando..." : "Tweetar"}</Text>
</Pressable>
```


E DEPOIS?

```
{
  isError && (
    <Text>
      Não foi possível tweetar:
      {error.message}
    </Text>
  );
}

{
  isSuccess && <Text>Tweet adicionado com sucesso!</Text>;
}
```

O ESTADO DA MUTATION CONTROLA A INTERFACE.

UMA DIFERENÇA IMPORTANTE

`isPending` não significa:

“a aplicação está carregando”

Significa:

“**esta operação** ainda está sendo executada.”

E SE QUISERMOS REAGIR AO RESULTADO?

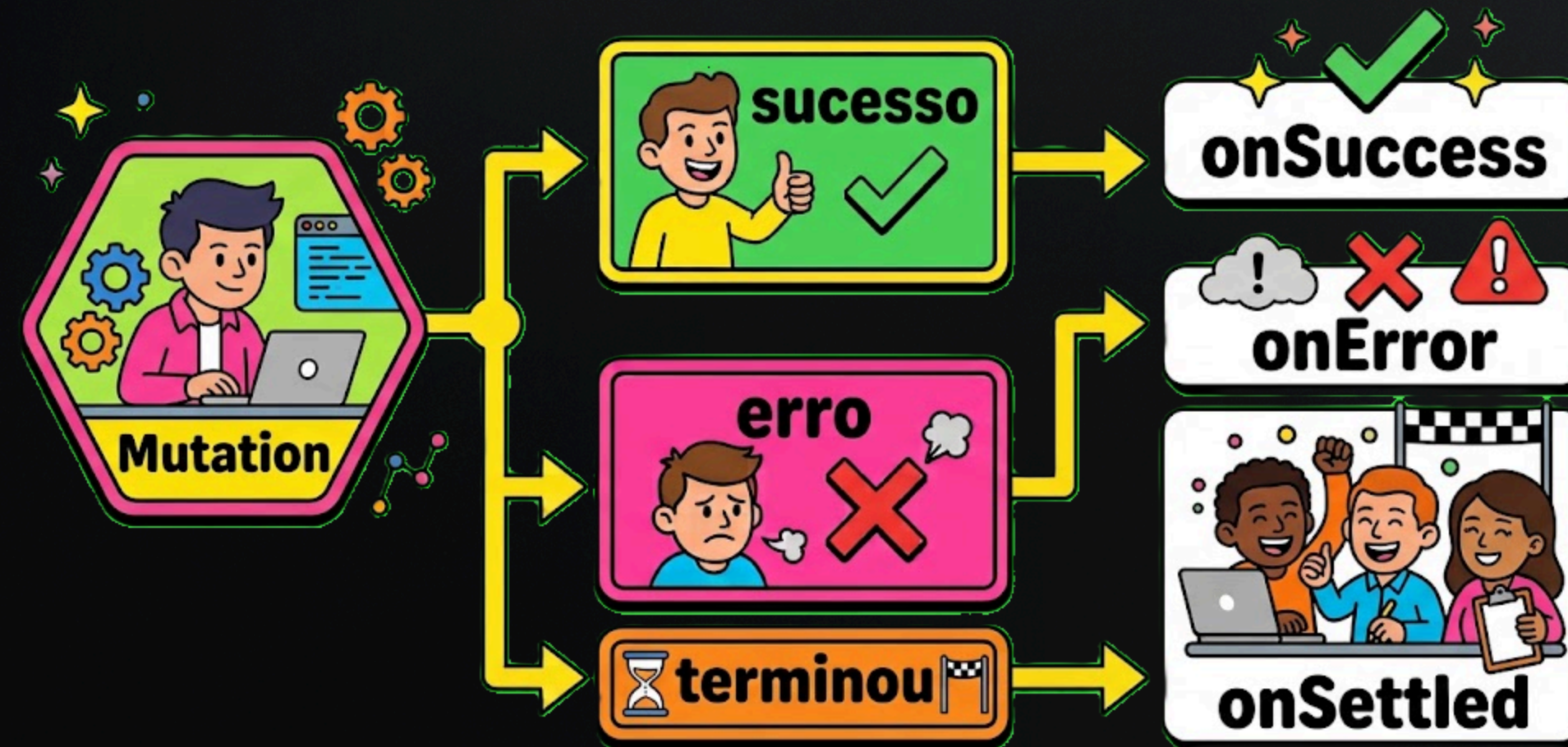
Por exemplo:

- limpar o formulário depois de salvar;
- mostrar uma mensagem;
- navegar para outra tela;
- registrar um erro.

SÃO CONSEQUÊNCIAS DA OPERAÇÃO.

MUTATION SIDE EFFECTS

- `useMutation` permite reagir ao ciclo da mutation:



ONSUCCESS

```
const mutation = useMutation({
  mutationFn: createTweet,

  onSuccess: () => {
    setText("");
  },
});
```

A API continua sendo responsabilidade da:

mutationFn;

O que fazer **depois do sucesso** fica no:

onSuccess;

ONERROR

```
const mutation = useMutation({  
  mutationFn: createTweet,  
  
  onError: () => {  
    Alert.alert("Erro", "Não foi possível tweetar.");  
  },  
});
```


ONSETTLED

É executado quando a mutation termina:



Não importa se houve sucesso ou erro.

QUEM FAZ O QUÊ?

`mutationFn`

Executa a operação.

```
criar  
editar  
excluir
```

Callbacks

Reagem ao resultado.

```
onSuccess  
onError  
onSettled
```


DESAFIO

Nossa lista possui tweets.

Adicione uma ação para:

+ **EXCLUIR UM TWEET**

REQUISITOS

A solução deve:

- criar uma função para excluir o recurso da API;
- usar `useMutation`;
- receber o `id` através de `mutate(id)`;
- impedir novos cliques enquanto `isPending`;
- apresentar feedback em caso de erro.

NÃO TENHA TENTADO ATUALIZAR A LISTA MANUALMENTE.



THINKING IN SYSTEMS

DONELLA H. MEADOWS

Uma mudança raramente existe isoladamente.

Alterar uma parte de um sistema pode produzir consequências em outras partes que dependem daquele estado.

Uma boa introdução para começar a enxergar software — e problemas fora dele — como sistemas de relações, estados e feedback.

LICENÇA

Este material é disponibilizado sob a licença Creative Commons Atribuição 4.0 Internacional (CC BY 4.0).

  Atribuição 4.0 Internacional

Autor: Felipe Cavalcante Vilella de Oliveira, 2026